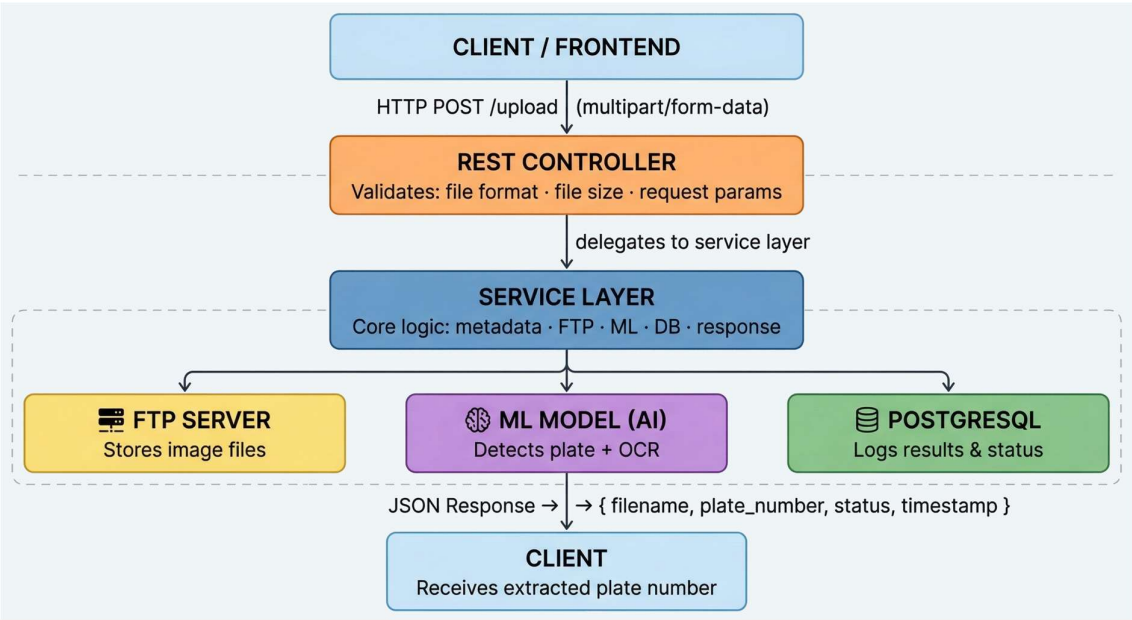
Number Plate Detection Project

In this Project the application combines a Java Spring Boot microservice backend, an FTP-based file storage layer, a PostgreSQL audit database, and a Python-based machine learning inference pipeline. The system exposes RESTful HTTP APIs that accept vehicle images, run them through a YOLO/OpenCV detection model, and return annotated output images with bounding boxes and text labels overlaid on the detected number plates.

Architecture



Components Explanation

REST Controller *Spring Boot @RestController*
The Controller is the front door of the backend. It receives every HTTP request, checks that the uploaded file is a valid image (JPEG or PNG), verifies the file size is within limits, and confirms all required headers and parameters are present. If anything is wrong, it immediately returns a descriptive HTTP 400 Bad Request error — nothing invalid ever reaches the service layer. This keeps the rest of the pipeline clean and predictable.

Service Layer *Spring Boot @Service*
This is the core of the application. It coordinates the entire pipeline: generating a unique filename and timestamp, verifying FTP and database connectivity, uploading the image to the FTP server, invoking the Python detection script via a Java ProcessBuilder call, collecting the output, and persisting the result to the database. It is the only layer that knows how all the other parts fit together.

FTP Server *Windows IIS — Port 21*
Images are stored on a dedicated Windows IIS FTP server rather than on the application server itself. The service layer uses Apache Commons Net (FTPClient) to connect, recursively create any missing subdirectory structure, and upload the file in binary mode. Keeping storage separate from the application reduces server load and allows images to be accessed and archived independently.

ML Detection Model *Python + YOLO + OpenCV*
The Python script predict.py handles all AI inference. It loads the uploaded image using cv2.imread(), runs it through a trained YOLO model to detect the number plate region and draw a bounding box around it, then applies OCR to extract the alphanumeric characters (e.g. KL07AB1234). The annotated image is saved and re-uploaded to the FTP server alongside the original.

PostgreSQL Database *Hibernate ORM / Spring Data JPA*
Every request — successful or failed — generates a database record. Hibernate ORM maps Java objects directly to database rows, so no raw SQL is needed. The audit table records the filename, FTP path, extracted plate text, processing status, and full server logs. This history is invaluable for debugging, usage analytics, and operational auditing.

Working Sequence

1 **Client Request**
The user selects or captures a vehicle image on the frontend. The image is sent to the backend as multipart/form-data via HTTP POST to the /upload endpoint. The request also carries metadata such as the desired folder path in a custom HTTP header (X-Ftp-Path).

2 **Controller Validation**
The Spring Boot REST Controller is the first layer to receive the request. It checks that the uploaded file is a valid image format (JPEG/PNG), is within the permitted file size limit, and that all required request parameters are present. Any request that fails these checks is rejected immediately with a descriptive HTTP 400 error — preventing wasted processing downstream.

3 **Service Layer Processing**
Once validated, the Controller hands the request to the Service Layer, which is the brain of the backend. It generates a unique filename and timestamp for the upload, checks FTP server connectivity and database availability, and then coordinates the full pipeline: upload → detect → log → respond.

4 **FTP Storage**
The image is uploaded to a centralized Windows IIS FTP server. The service layer creates any missing subdirectories recursively before storing the file. Keeping images on a dedicated FTP server offloads storage from the application server and allows images to be retrieved independently by other services or for archiving.

5 **ML Detection + OCR**
The service layer triggers the Python-based detection script (predict.py) using a Java ProcessBuilder call. The script loads the image, runs it through a YOLO object detection model to find the number plate region, draws a bounding box around it, then applies OCR to extract the alphanumeric text (e.g., KL07AB1234). The annotated output image is saved and re-uploaded to the FTP server.

6 **Database Logging**

Every detection event — successful or not — is recorded in PostgreSQL via Hibernate ORM. The record includes the filename, FTP path, extracted plate text, processing status (success/failure), and any server logs. This audit trail is critical for debugging, analytics, and compliance.

7 JSON Response

Once all processing is complete, a structured JSON response is sent back to the client containing the filename, detected plate number, overall status, and the upload timestamp. The frontend can display or use this data immediately — no polling required.